CineBook

Online Movie Ticket Booking System

Submitted By -

Harshith AP23110010492

Ashwin AP23110010847

Vittal AP23110010231

Vaishnavi AP23110010807

Pravallika AP23110010520

Submitted To –

Ajay Sir

INTRODUCTION:

CineBook is a full-stack web application designed to simplify and modernize the process of booking movie tickets online. It provides users with a seamless experience to browse currently available movies, explore show details, choose their preferred theater and timing, select seats, and complete their bookings — all from a single platform.

Traditionally, moviegoers had to either visit a cinema counter or rely on third-party apps that were often cluttered and slow. CineBook addresses this problem by offering a clean, fast, and user-friendly interface that puts the entire booking journey in the user's hands.

On the administrative side, CineBook includes a comprehensive dashboard that allows admins to manage every aspect of the system — from adding new movies and theaters to scheduling shows and setting ticket prices. The system is built using the MERN stack (MongoDB, Express.js, React.js, Node.js), making it scalable, maintainable, and productionready.

OBJECTIVE:

The key objectives of the CineBook project are:

- To build a reliable online platform where users can browse movies and book tickets without visiting a physical counter.
- To implement a real-time seat selection system that shows Available, Selected, Booked, and Locked seat states.
- To allow users to filter movies by city, theater, and show timings for better personalization.
- To provide a secure authentication system (Login and Signup) for users.
- To design an Admin Dashboard that gives full control over movies, cities, theaters, and shows.
- To create a scalable backend using RESTful APIs that can support future growth.
- To deliver a responsive and visually appealing frontend using React.js and Tailwind CSS.

SYSTEM REQUIREMENTS:

To ensure smooth development, deployment, and usage of the CineBook Web Application, certain system prerequisites must be met. These requirements are categorized into software and hardware specifications.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:**
  Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v18 or above):**

Provides the runtime environment for building and managing server-side logic and API routing.

- **npm (v8 or above):**
  A package manager required to install dependencies for React and related libraries.
- **React.js:**
  A JavaScript library for building dynamic and responsive user interfaces.
- **Tailwind CSS:**
  A utility-first CSS framework for designing modern, responsive UIs.
- **Express.js:**
  A lightweight web framework for building RESTful APIs.
- **MongoDB:**
  A NoSQL database used to store all application data including users, movies, theaters, and bookings.
- **Postman:**
  Tool for testing APIs during development.
- **Visual Studio Code:**
  Preferred code editor with built-in Git and terminal support.
- **Browser:**
  Google Chrome / Firefox (latest version) — For rendering and testing the UI in realtime.

2. Hardware Requirements

Describes the minimum specifications needed to support the development and usage of the application.

- **Processor:**
  Intel Core i3 (or equivalent) — Ensures smooth compilation and multitasking during development.
- **RAM:**
  Minimum 4 GB (8 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:**
  At least 500 MB free space — Required for package installations, MongoDB setup, and local project files.
- **Network:**
  Stable internet connection for MongoDB Atlas / API calls.
- **Display:**
  1280x720 resolution or higher — Recommended for optimal coding experience and UI visualization.

PROJECT ARCHITECTURE:

CineBook follows a three-tier client-server architecture, which separates the system into three distinct layers: Presentation, Business Logic, and Data.

Presentation Layer (Frontend)

Built with React.js and styled using Tailwind CSS. The frontend is a Single Page Application (SPA) that renders different views without full page reloads. It communicates with the backend through REST API calls. All user interactions — browsing movies, selecting seats, and completing bookings — happen here.

Business Logic Layer (Backend)

Built using Node.js with Express.js. The backend handles incoming HTTP requests, applies business rules (seat locking, booking validation, JWT authentication), and processes data before sending responses. It exposes a set of RESTful API endpoints consumed by the frontend.

Data Layer (Database)

MongoDB is used as the NoSQL database. Mongoose is used as an ODM (Object Data Modeling) library to define schemas and interact with the database. All data — users, movies, theaters, shows, seats, and bookings — is stored in MongoDB collections.

Architecture Flow (Textual Representation)

```
[User Browser]
       |
       |  HTTP Request (REST API)        v
[React.js Frontend  —  Tailwind CSS]
       |
       |  Axios / Fetch API Calls        v
[Node.js + Express.js Backend]
       |  Route  -->  Controller  -->  Model        v
[MongoDB Database]
```

TECHNICAL ARCHITECTURE:

The technical architecture of CineBook is based on the request-response cycle of RESTful APIs. Here is how a typical interaction works in the system:

1. The user performs an action on the React frontend (e.g., clicks 'Book Now').
2. React sends an HTTP request (GET / POST / PUT / DELETE) to the Express.js backend.
3. The request hits the appropriate route defined in the backend routing layer.
4. The route calls the corresponding controller function to handle business logic.
5. The controller interacts with the Mongoose model to read or write data in MongoDB.
6. MongoDB returns the result to the controller, which formats a JSON response.
7. The JSON response is sent back to the React frontend, which updates the UI accordingly.

Authentication Flow

JWT (JSON Web Token) based authentication is used in CineBook:

- On login or signup, the server generates a JWT and sends it to the client.
- The client stores the token in localStorage.
- For protected routes (e.g., booking), the token is sent in the Authorization request header.
- The server verifies the token using middleware before processing the request.

Seat Locking Mechanism

When a user selects a seat, it is temporarily locked to prevent double booking. A seat can have four states: Available (open to all users), Selected (chosen by current user), Booked (already confirmed by another user), and Locked (temporarily reserved). The backend manages seat state updates to maintain data consistency.

ER DIAGRAM:

The Entity-Relationship (ER) diagram for CineBook represents the various entities and their relationships within the movie ticket booking system. Here is a description of the key components:

Entities:

1. User

Represents registered users and admins on the platform.

- Attributes: user_id (PK), name, email, password, role (user/admin), created_at

2. Movie

Stores all movie details displayed to users on the platform.

- Attributes: movie_id (PK), title, description, genre, language, duration, rating, poster_url, release_date

3. City

Represents cities where theaters are located.

- Attributes: city_id (PK), city_name, state

4. Theater

Each theater belongs to a specific city.

- Attributes: theater_id (PK), theater_name, city_id (FK), address, total_seats

5. Show

A show links a specific movie to a specific theater at a specific date and time.

- Attributes: show_id (PK), movie_id (FK), theater_id (FK), show_date, show_time, price, available_seats

6. Seat

Each show has a set of seats, each with a current booking status.

- Attributes: seat_id (PK), show_id (FK), seat_number, row, status (Available / Selected /

Booked / Locked)

7. Booking

Records a completed booking made by a user for a particular show.

- Attributes: booking_id (PK), user_id (FK), show_id (FK), seat_ids (array), total_price, booked_at, status

Relationships:

- User — Booking: One user can place many bookings. (1:N)
- Movie — Show: One movie can have many shows. (1:N)
- City — Theater: One city can have many theaters. (1:N)
- Theater — Show: One theater can host many shows. (1:N)
- Show — Seat: One show has many seats. (1:N)
- Booking — Seat: One booking can reserve one or more seats. (1:N)

Attributes:

Each entity has its own set of attributes that describe its properties. For example, the Movie entity has attributes such as movie_id, title, genre, language, duration, and poster_url.

Primary Keys:

Each entity has a primary key that uniquely identifies each record. For example, the Movie entity's primary key is movie_id, and the Booking entity's primary key is booking_id.

Foreign Keys:

Foreign keys are used to establish relationships between entities. For example, the Show entity has foreign keys referencing movie_id and theater_id to link a show to a specific movie and theater. The Booking entity has foreign keys referencing user_id and show_id.

Overall, the ER diagram for CineBook provides a clear representation of the database schema, showing how different entities are related and how data flows between them in the movie ticket booking system.

KEY FEATURES:

The key features of CineBook can be divided into User Features and Admin Features, covering the complete functionality of the platform.

User Features:

- **User Registration and Login:**
  Allows users to create accounts and log in securely using JWT-based authentication.
- **Browse Movies:**
  View all available movies on the landing page with poster cards.
- **Movie Details Page:**
  View full details of a selected movie including genre, language, rating, and description.

- **City and Theater Selection:**
  Choose a preferred city from the available list and view theaters in that city.
- **Show Timings:**
  View available dates and time slots for each movie at the selected theater.
- **Interactive Seat Selection:**
  Visual seat grid with real-time status for each seat — Available, Selected, Booked, and Locked.
- **Booking Confirmation:**
  Complete a booking and receive a summary with movie, show, seats, and total price details.
- **Responsive Design:**
  Optimized for both desktop and mobile browsers.

Admin Features:

- **Admin Dashboard:**
  A dedicated control panel for managing all platform data.
- **Movie Management:**
  Add, edit, or delete movies with all relevant details and poster image.
- **City Management:**
  Add and manage cities where theaters are available.
- **Theater Management:**
  Add theaters and link them to specific cities.
- **Show Management:**
  Assign shows by selecting a movie, theater, date, time, and ticket price.
- **Data Overview:**
  View and manage all bookings and system data from a centralized dashboard.

ROLES AND RESPONSIBILITIES:

User:

- **Registration and Account Management:**
  Users are responsible for creating an account, providing accurate personal information, and managing their account details including profile updates and password security.
- **Movie Browsing and Booking:**
  Users can browse available movies, select a city and theater, choose a show timing, select seats from the interactive grid, and complete the booking.
- **Seat Selection:**
  Users are responsible for selecting their desired seats from the available seats in the seat map and confirming their selection before proceeding to booking.

- **Booking Confirmation:**
  Users must review the booking summary (movie, show, seats, total price) and confirm the booking to finalize their ticket reservation.

Admin:

- **System Management:**
  The admin is responsible for overall management of the CineBook platform. This includes monitoring system performance, ensuring data accuracy, and managing all platform settings.
- **Movie and Content Management:**
  The admin adds new movies, updates existing movie details, and removes movies that are no longer showing.
- **Theater and City Management:**
  The admin manages cities and theaters on the platform, ensuring the accuracy and quality of theater data listed for users.
- **Show Scheduling:**
  The admin creates and manages shows by linking movies to theaters with specific dates, times, and pricing.

USER FLOW:

Customer Flow

- **Registration and Account:**
  Create and manage account details securely.
- **Browse Movies:**
  Explore movies on the landing page and view full details by clicking on a movie card.
- **Show Selection:**
  Choose city, theater, date, and time slot for the movie.
- **Seat Selection:**
  Select desired seats from the interactive grid and proceed to booking.
- **Booking Confirmation:**
  Review and confirm the booking summary — seats are marked as Booked.

Admin Flow

- **System Management:**
  Oversee platform performance, settings, and data integrity.
- **Content Management:**
  Add, edit, or remove movies, cities, and theaters.
- **Show Management:**
  Create and schedule shows by assigning movies to theaters with date, time, and pricing.

MVC PATTERN:

The CineBook application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB. In CineBook, models include User, Movie, City, Theater, Show, Seat, and Booking.

Controller Layer

The Controller layer acts as an intermediary between the View (routes) and the Model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client. For example, the BookingController handles seat validation, booking creation, and seat status updates.

View Layer (Routing Layer)

In the context of the CineBook backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions. On the frontend, React.js components serve as the View, rendering UI for users.

Advantages of Using MVC in This Project

- **Separation of Concerns:**
  Each layer has a clearly defined responsibility, improving readability and maintainability.

Scalability:

New features can be added easily by creating new routes, controllers, and models.

- **Reusability:**
  Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:**
  Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly:**
  Multiple developers can work simultaneously on different layers without conflict.

PROJECT SETUP AND CONFIGURATION:

Follow these steps to set up and run the CineBook application on your local machine.

Creating the Project Folder:

8. Create a new folder with your project name (e.g., CineBook).
9. Inside that folder, create two subfolders.
10. Name one as Client.
11. Name the other as Server.
12. Open the project folder in VS Code.

Client Setup (Installing React App)

Open the Client folder in the terminal of VS Code and run the following commands:

```
npm create vite@latest . -- --template react
```

Select React as the framework and JavaScript as the variant.

```
cd client npm
install
npm install -D tailwindcss postcss autoprefixer npm
run dev
```

The React app will run at: http://localhost:5173

Server Setup (npm init)

Open the Server folder in the terminal of VS Code and run:

```
npm init -y npm install express mongoose dotenv jsonwebtoken bcryptjs
cors npm install -D nodemon
```

Create the following file and folders inside the Server directory:

13. server.js — Main entry point for the Express application.
14. models/ — Folder for all Mongoose schema files.
15. controllers/ — Folder for all business logic functions.
16. routes/ — Folder for all API route definitions.
17. middleware/ — Folder for JWT auth middleware and error handlers.
18. .env — Environment variables file (MongoDB URI, JWT Secret).

Start the server using: npm run dev. The backend API will run at: http://localhost:5000

BACKEND DEVELOPMENT:

Backend Structure:

Controllers

- **authController.js**
  Handles user registration (hashes password using bcrypt) and login (issues JWT token).
- **movieController.js**
  Handles fetching all movies, a single movie by ID, and adding new movies (admin only).
- **showController.js**
  Fetches available shows by movie and theater and handles creation of new shows.
- **bookingController.js**
  Validates seat availability, creates booking records, and updates seat status to Booked.
- **adminController.js**
  Handles all admin-specific operations including city, theater, and show management.

Middleware

- **authMiddleware.js**
  Middleware for authentication and authorization — verifies JWT token on protected routes.
- **errorHandler.js**
  Global error handling middleware for consistent API error responses.

Models

- **User.js**
  Schema/model for user accounts (name, email, password, role).
- **Movie.js**
  Schema/model for movie details (title, genre, language, duration, poster_url).
- **City.js**
  Schema/model for city data.
- **Theater.js**
  Schema/model for theater details linked to a city.
- **Show.js**
  Schema/model for shows linking a movie, theater, date, time, and price.
- **Seat.js**
  Schema/model for individual seats with status tracking per show.

Booking.js

Schema/model for confirmed bookings with references to user, show, and seats.

Routes

- **authRoutes.js**
  Defines API routes for /api/auth/register and /api/auth/login.
- **movieRoutes.js**
  Defines API routes for fetching and managing movies.
- **showRoutes.js**
  Defines API routes for fetching and creating shows.
- **bookingRoutes.js**
  Defines API routes for creating and fetching bookings.
- **adminRoutes.js**
  Defines API routes for all admin actions (add movies, cities, theaters, shows).

DATABASE DEVELOPMENT:

Configure MongoDB:

- Install Mongoose. In your Node.js project directory, run: npm install mongoose
  Create database connection using the MONGO_URI from your .env file.
- Create Schemas and Models based on the ER diagram design.

Create Database Connection:

Make sure the database is connected before performing any actions through the backend. The connection is established in a db/config.js file using mongoose.connect(), which reads the MONGO_URI from environment variables. This function is called once when the server starts.

Create Schemas:

Configure the Schemas for the MongoDB database to store data in a structured pattern. Use the data from the ER diagrams to create the schemas. The schemas for this application are:

19. **1. User.js**
    Schema/model for user accounts. Fields: name, email, password (hashed), role.
20. **2. Movie.js**
    Schema/model for movie details. Fields: title, description, genre, language, duration, rating, poster_url, release_date.
21. **3. City.js**
    Schema/model for city data. Fields: city_name, state.
22. **4. Theater.js**
    Schema/model for theater details. Fields: theater_name, city (ref: City), address, total_seats.

23. **5. Show.js**
    Schema/model for shows. Fields: movie (ref: Movie), theater (ref: Theater), show_date, show_time, price, available_seats.
24. **6. Seat.js**
    Schema/model for individual seat records per show. Fields: show (ref: Show), seat_number, row, status.
25. **7. Booking.js**
    Schema/model for completed bookings. Fields: user (ref: User), show (ref: Show), seats (array), total_price, booked_at, status.

FRONT-END DEVELOPMENT:

<u>Frontend Structure:</u>

Admin (src/Admin)

- **AdminDashboard.jsx**
  Main admin panel page with sidebar navigation to all admin sections.
- **AdminMovies.jsx**
  UI for admin to view, add, edit, and delete movies.
- **AdminCities.jsx**
  Admin panel to manage cities.
- **AdminTheaters.jsx**
  Admin panel to manage theaters linked to cities.
- **AdminShows.jsx**
  Admin panel for creating and managing shows (movie + theater + date + time + price).
- **AdminLogin.jsx** Admin login page.

User (src/User)

- **LandingPage.jsx**
  Homepage with recommended movies, hero banner, and movie grid.
- **MovieDetailPage.jsx**
  Full detail view for a selected movie (poster, description, Book Now button).
- **ShowSelectionPage.jsx**
  Displays available dates, theaters, and time slots for a movie.
- **SeatSelectionPage.jsx**
  Interactive seat map with real-time status for each seat in grid layout (A to J rows).
- **BookingConfirmation.jsx**
  Summary of the booking with movie, show, seat details, and total price.

Login.jsx / Signup.jsx

Authentication forms for user registration and login.

Components (Reusable Parts)

- **Navbar.jsx**
  Top navigation bar with logo, links, and login/logout button.
- **MovieCard.jsx**
  Reusable card component displaying movie poster, title, and action buttons (View Details / Book Now).
- **SeatGrid.jsx**
  Grid layout (rows A to J) rendering each seat as a colored button based on its status.
- **Footer.jsx**
  Footer component used across all pages.
- **ProtectedRoute.jsx**
  HOC that restricts access to authenticated users and admins only.

OUTPUT SCREENS:

Landing Page:

The landing page is the first screen users see when they visit CineBook. It features a hero banner section at the top with the CineBook branding and a tagline. Below the banner, a grid of movie cards is displayed — each showing the movie poster, title, genre, and two action buttons: 'View Details' and 'Book Now'. The navigation bar at the top includes the CineBook logo on the left and Login / Signup buttons on the right.

Movie Detail Page:

Clicking on a movie card opens the Movie Detail Page. This screen shows the full movie poster on the left side, and on the right side displays the movie title, description, genre, language, duration, release date, and rating. A prominent 'Book Tickets' button at the bottom guides users to the show selection step.

Show Selection Page:

After choosing to book, the user lands on the Show Selection Page. Here they can select a city from a dropdown, view theaters available in that city, and choose a specific date and time slot for the show. Available shows are displayed as interactive cards, each showing the theater name, show time, and ticket price.

Seat Selection UI:

The Seat Selection screen is one of the most interactive features of CineBook. It displays a grid layout of seats organized in rows A through J (10 rows, 10 seats per row = 100 seats total). Each seat is color-coded: Green for Available, Yellow for Selected by current user, Gray for Booked, and Orange for Locked. A legend at the top explains each color. Users click on available

seats to select them. Below the grid, the total ticket count and total price are dynamically updated. A 'Proceed to Book' button completes the selection.

Admin Dashboard:

The Admin Dashboard is a multi-section control panel accessible only to admins. The sidebar provides navigation links to Movies, Cities, Theaters, and Shows sections. Each section displays a data table listing existing records with options to edit or delete. An 'Add New' button at the top of each section opens a form for creating new entries. The dashboard provides complete control over all content displayed to users.

Login / Signup Page:

Clean, centered authentication forms with fields for Full Name, Email Address, and Password. The Signup page includes a 'Sign Up' button and a link to Login for existing users. The Login page includes an 'Log In' button and a link to create a new account.

ADVANTAGES:

- **Convenience:**
  Users can book tickets anytime, anywhere without visiting a physical ticket counter.
- **Real-Time Seat Availability:**
  The seat grid updates in real time, preventing double bookings and providing accurate availability.
- **Scalable Architecture:**
  The MERN stack is highly scalable and can handle growing user loads without major rework.
- **Admin Control:**
  The admin dashboard provides complete control over movies, theaters, cities, and shows.
- **Secure Authentication:**
  JWT-based authentication ensures that only authorized users can book tickets and only admins can manage data.
- **Clean and Responsive UI:**
  Tailwind CSS ensures a responsive, modern, and consistent interface across all screen sizes.
- **Organized Codebase:**
  The MVC pattern and component-based frontend architecture keep the code clean, modular, and easy to maintain.

LIMITATIONS:

- **No Payment Gateway:**

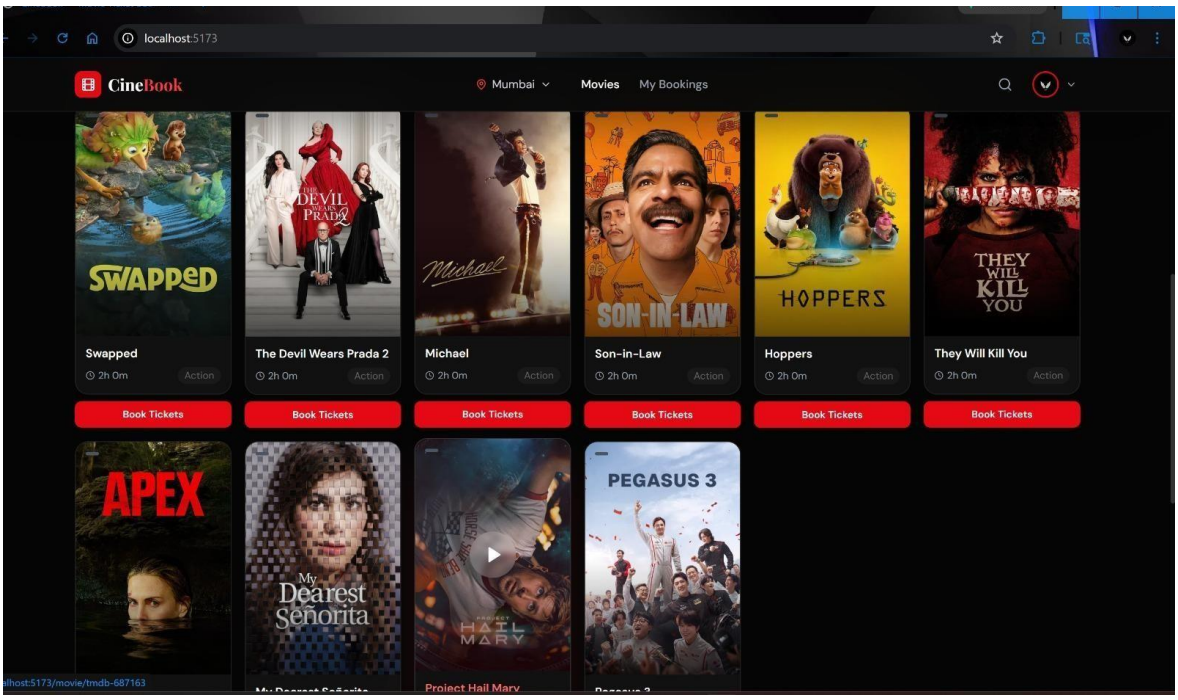
The current version does not integrate a real payment system such as Razorpay or Stripe.

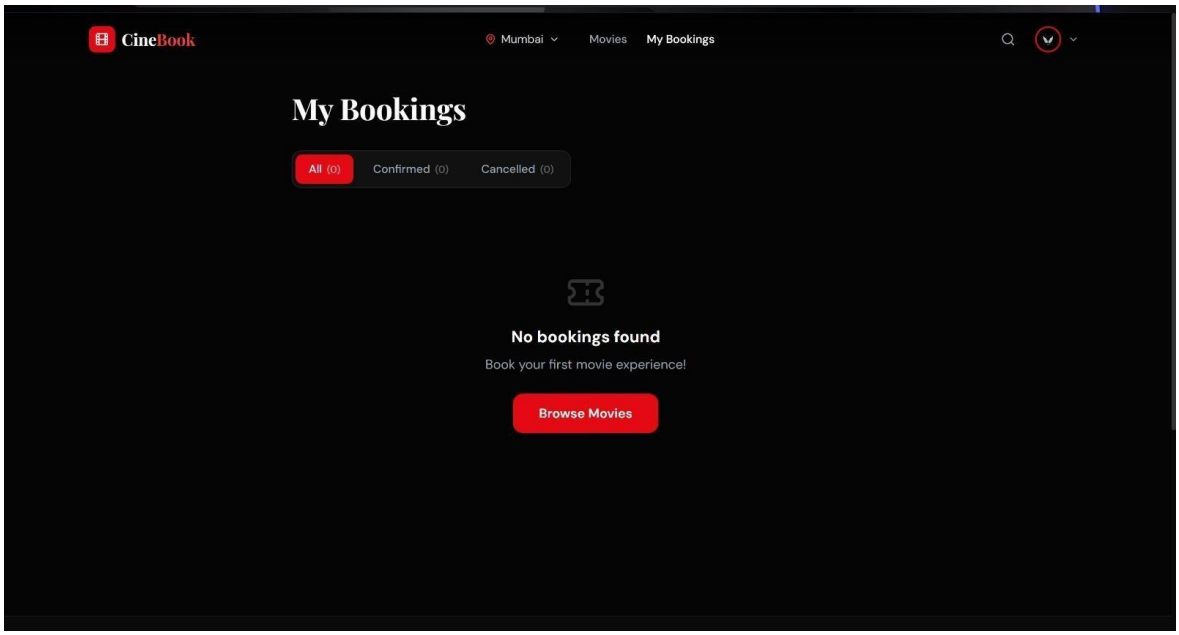
- **No Email Notifications:**
  Users do not receive email or SMS confirmations after completing a booking.
- **No Advanced Search:**
  Advanced movie search and filtering by genre, language, or rating is not yet implemented.
- **No Ticket Cancellation:**
  Users cannot cancel a booked ticket from within the application.
- **No WebSocket Integration:**
  Seat status updates are not WebSocket-based, which may cause minor delays in multiuser scenarios.
- **Basic Role System:**
  There is only one admin role. Role-based access control (RBAC) with multiple permission levels is not implemented.

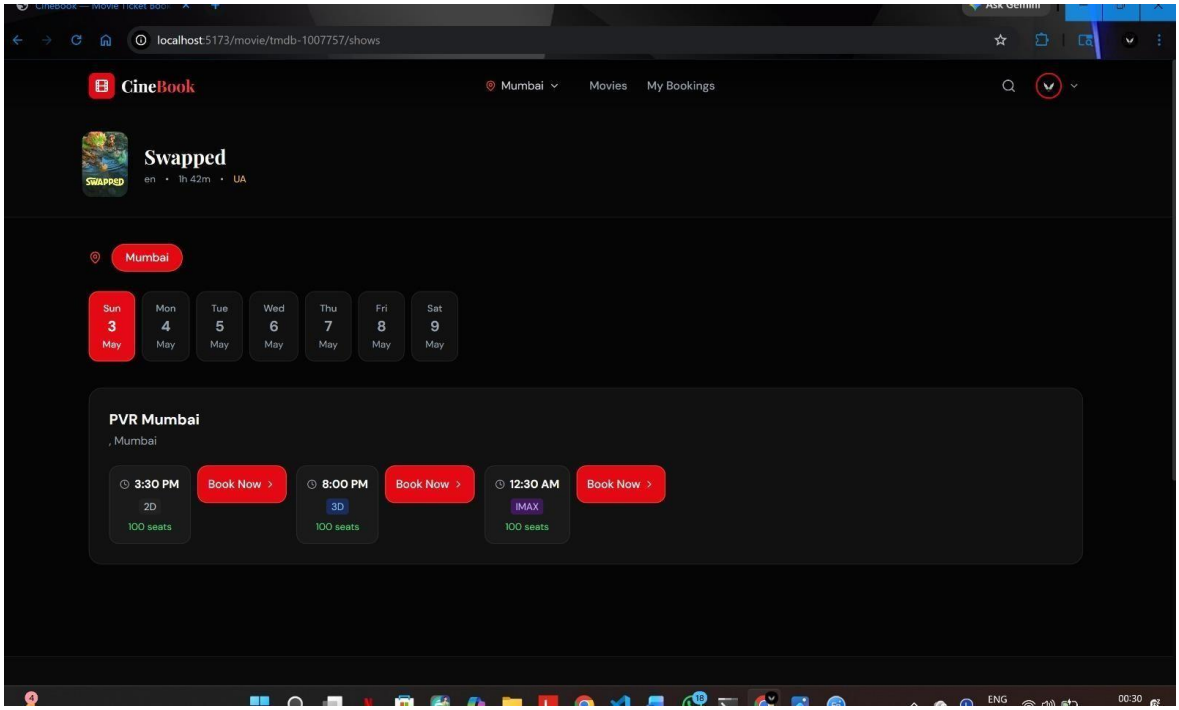
FUTURE ENHANCEMENTS:

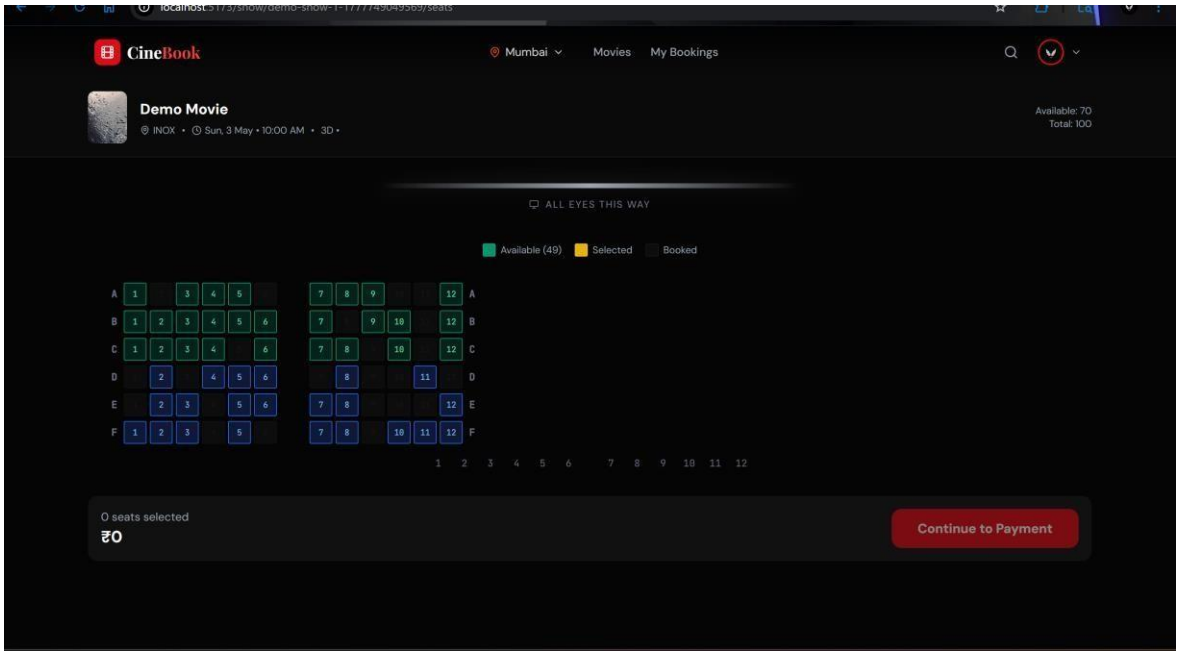
- **Payment Integration:**
  Integrate a payment gateway such as Razorpay or Stripe to enable real online transactions.
- **Email / SMS Notifications:**
  Send booking confirmations and reminders via email or SMS using Nodemailer or Twilio.
- **WebSocket Integration:**
  Use Socket.io to provide real-time seat status synchronization across multiple users simultaneously.
- **Search and Filters:**
  Add search functionality by title and filters by genre, language, and rating.
- **Ticket Cancellation:**
  Allow users to cancel bookings within a defined time window with automated refund processing.
- **User Profile and Booking History:**
  Let users view past bookings and download e-tickets from their profile page.
- **Movie Ratings and Reviews:**
  Allow users to rate and review movies after watching.
- **Mobile Application:**
  Develop a React Native app for iOS and Android using the same backend APIs.

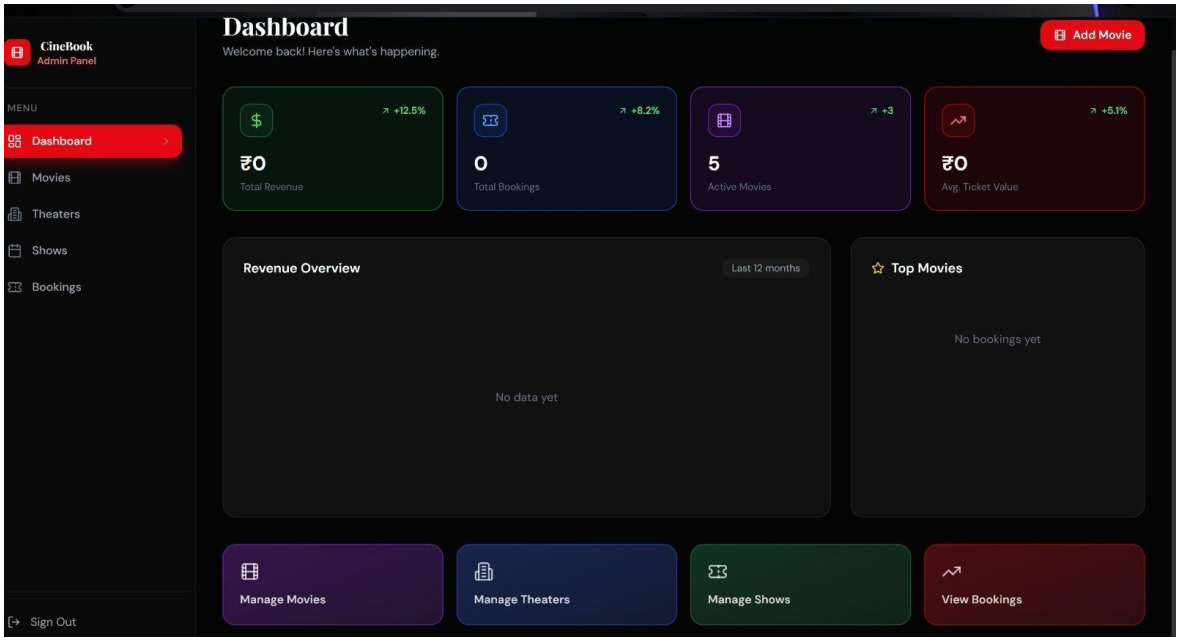
OUTPUT SCREENSHOTS:

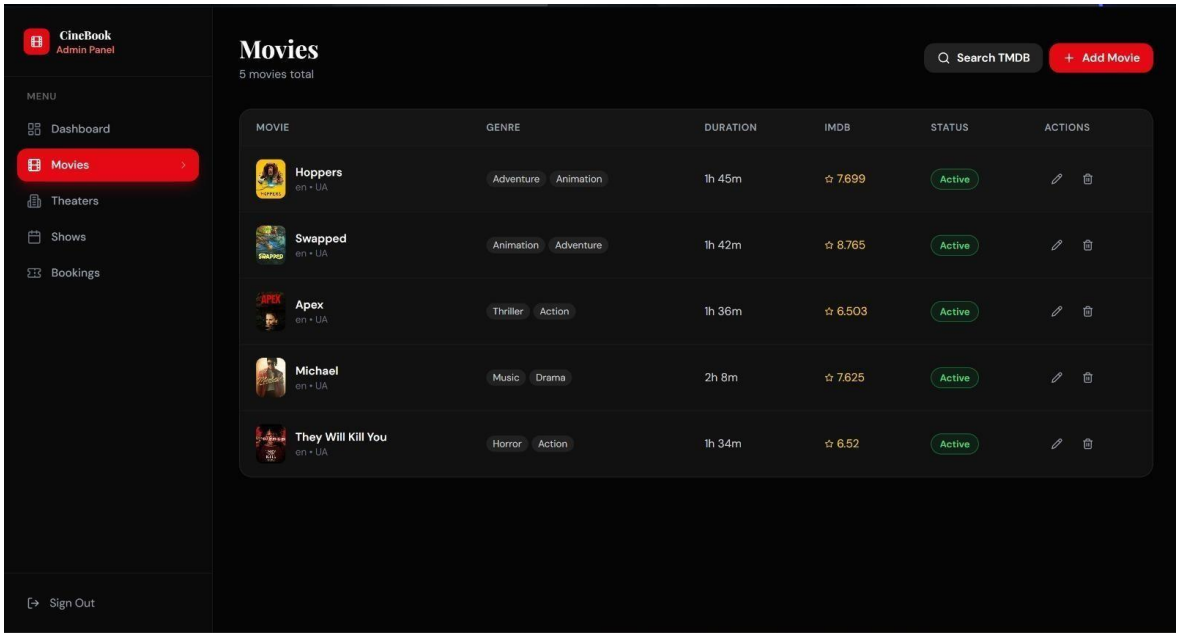


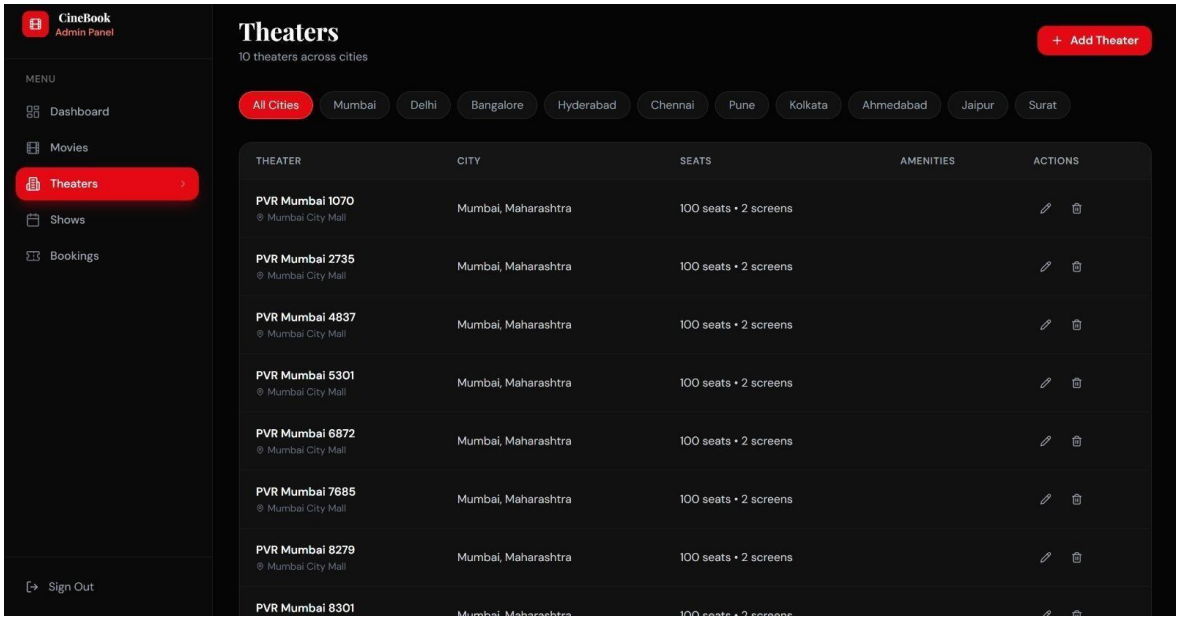

CineBook
Mumbai
Movies
My Bookings
My Bookings
All (0)
Confirmed (0)
Cancelled (0)
No bookings found
Book your first movie experience!
Browse Movies

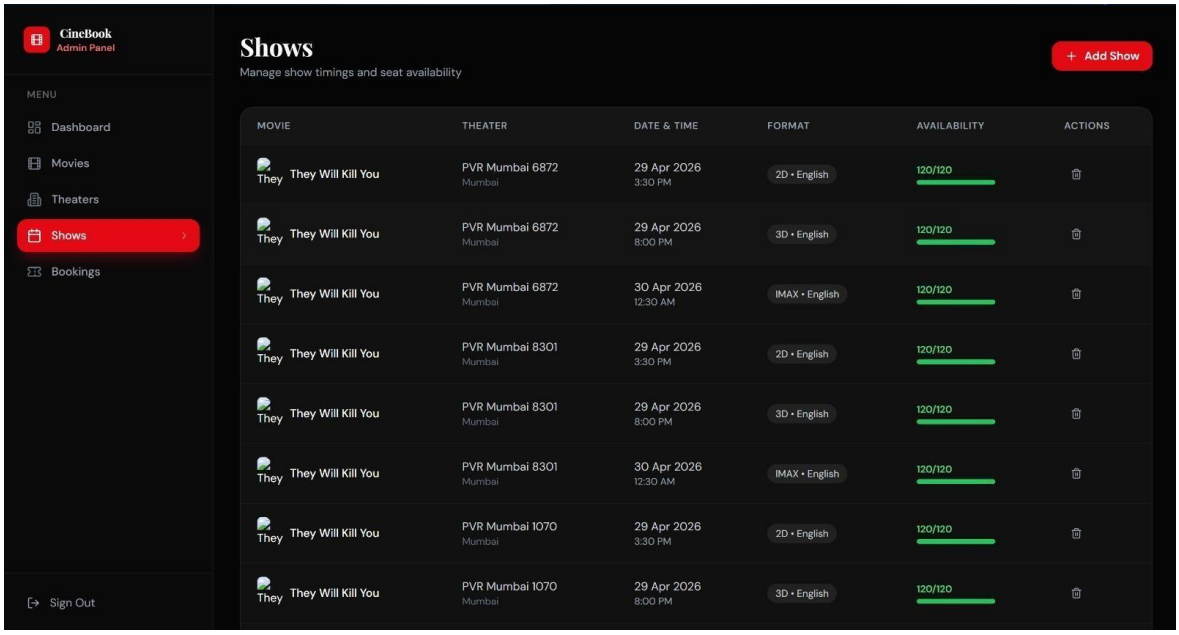

localhost:5173/movie/tmdb-1007757/shows
CineBook
Mumbai
Movies
My Bookings
Swapped
en • 1h 42m • UA
Mumbai
Sun 3 May
Mon 4 May
Tue 5 May
Wed 6 May
Thu 7 May
Fri 8 May
Sat 9 May
PVR Mumbai
, Mumbai
3:30 PM
2D
100 seats
Book Now
8:00 PM
3D
100 seats
Book Now
12:30 AM
IMAX
100 seats
Book Now

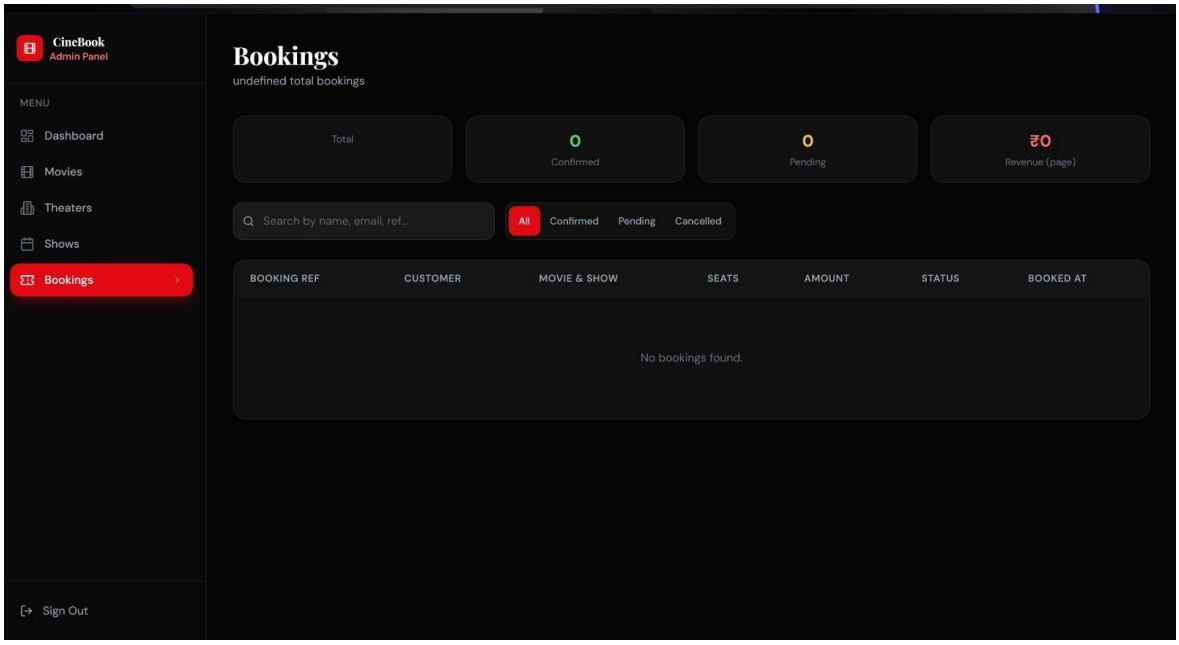

CineBook
Mumbai
Movies
My Bookings
Demo Movie
INOX • Sun, 3 May • 10:00 AM • 3D •
Available: 70
Total: 100
ALL EYES THIS WAY
Available (49)
Selected
Booked
0 seats selected
₹0
Continue to Payment

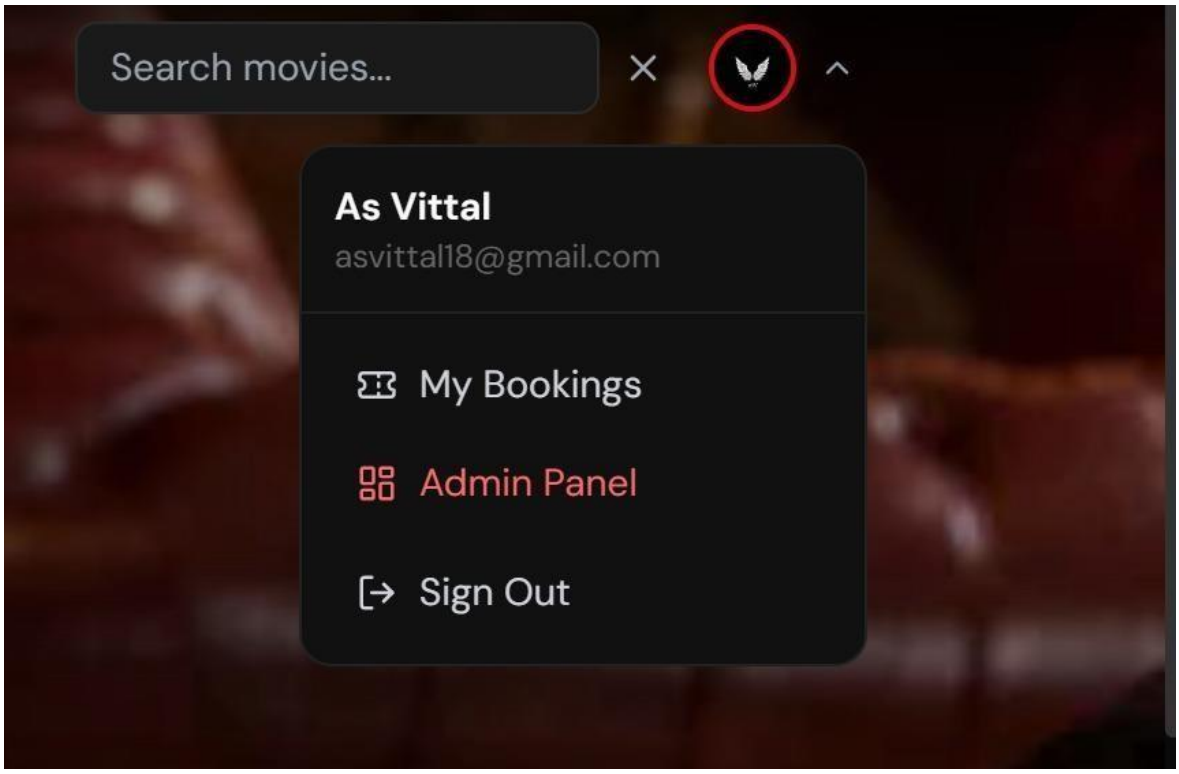

CineBook
Admin Panel
MENU
Dashboard
Movies
Theaters
Shows
Bookings
Sign Out
Dashboard
Welcome back! Here's what's happening.
Add Movie
+12.5%
₹0
Total Revenue
+8.2%
0
Total Bookings
+3
5
Active Movies
+5.1%
₹0
Avg. Ticket Value
Revenue Overview
Last 12 months
No data yet
Top Movies
No bookings yet
Manage Movies
Manage Theaters
Manage Shows
View Bookings


CineBook
Admin Panel
MENU
Dashboard
Movies
Theaters
Shows
Bookings
Sign Out
Movies
5 movies total
Search TMDB
+ Add Movie
MOVIE
GENRE
DURATION
IMDB
STATUS
ACTIONS
Hoppers
en • UA
Adventure
Animation
1h 45m
7.699
Active
Swapped
en • UA
Animation
Adventure
1h 42m
8.765
Active
Apex
en • UA
Thriller
Action
1h 36m
6.503
Active
Michael
en • UA
Music
Drama
2h 8m
7.625
Active
They Will Kill You
en • UA
Horror
Action
1h 34m
6.52
Active


CineBook
Admin Panel
MENU
Dashboard
Movies
Theaters
Shows
Bookings
Sign Out
Theaters
10 theaters across cities
+ Add Theater
All Cities
Mumbai
Delhi
Bangalore
Hyderabad
Chennai
Pune
Kolkata
Ahmedabad
Jaipur
Surat
THEATER
CITY
SEATS
AMENITIES
ACTIONS
PVR Mumbai 1070
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 2735
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 4837
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 5301
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 6872
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 7685
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 8279
Mumbai City Mall
Mumbai, Maharashtra
100 seats • 2 screens
PVR Mumbai 8301


CineBook
Admin Panel
MENU
Dashboard
Movies
Theaters
Shows
Bookings
Sign Out
Shows
Manage show timings and seat availability
+ Add Show
MOVIE
THEATER
DATE & TIME
FORMAT
AVAILABILITY
ACTIONS
They Will Kill You
PVR Mumbai 6872
Mumbai
29 Apr 2026
3:30 PM
2D • English
120/120
They Will Kill You
PVR Mumbai 6872
Mumbai
29 Apr 2026
8:00 PM
3D • English
120/120
They Will Kill You
PVR Mumbai 6872
Mumbai
30 Apr 2026
12:30 AM
IMAX • English
120/120
They Will Kill You
PVR Mumbai 8301
Mumbai
29 Apr 2026
3:30 PM
2D • English
120/120
They Will Kill You
PVR Mumbai 8301
Mumbai
29 Apr 2026
8:00 PM
3D • English
120/120
They Will Kill You
PVR Mumbai 8301
Mumbai
30 Apr 2026
12:30 AM
IMAX • English
120/120
They Will Kill You
PVR Mumbai 1070
Mumbai
29 Apr 2026
3:30 PM
2D • English
120/120
They Will Kill You
PVR Mumbai 1070
Mumbai
29 Apr 2026
8:00 PM
3D • English
120/120


CineBook
Admin Panel
MENU
Dashboard
Movies
Theaters
Shows
Bookings
Sign Out
Bookings
undefined total bookings
Total
0
Confirmed
0
Pending
₹0
Revenue (page)
Search by name, email, ref...
All
Confirmed
Pending
Cancelled
BOOKING REF
CUSTOMER
MOVIE & SHOW
SEATS
AMOUNT
STATUS
BOOKED AT
No bookings found.



CONCLUSION:

CineBook is a fully functional web application that successfully demonstrates the integration of modern web technologies to solve a real-world problem. By bringing the movie ticket booking experience online, it eliminates the need for physical counters and long queues, offering users a faster, more convenient way to plan their cinema visits.

The project covers all major aspects of a software engineering application — secure authentication, a dynamic and interactive frontend, a RESTful backend, a well-designed database, and a comprehensive admin management system. It follows best practices such as the MVC pattern, component-based UI design, and schema-based data modeling using Mongoose.

Through this project, practical knowledge was gained in building full-stack applications using the MERN stack, handling real-time data interactions, designing relational data in a NoSQL environment, and developing role-based user interfaces. CineBook serves as a strong foundation that can be extended into a production-grade system with the addition of payment processing, mobile support, and enhanced user features in the future.

--- End of Report ---